

CSE234 Week 13: CMake

CMake is the predominant build tool for C++ projects. There are other build tools as well (Meson, Premake, xmake, Bazel, etc.), but CMake is the most popular for C/C++.

Quick History Lesson: Make

Before CMake was `make`. Make is a build system for UNIX-like systems. It is very simple compared to CMake, but shares some similarities. Make (and CMake) use the last modified times of files to determine if a file needs to be recompiled. This allows Make to rebuild the minimum number of files required to update the program. Make uses a file called `Makefile` to configure the build for a project. Makefiles consist of *targets*, *dependencies*, *variables*, *comments*, *directives*, and *commands*. For our overview here, we are only concerned with targets, dependencies, and commands. Let's revisit the compilation of a basic three file program. To compile this, we need to compile the source code files separately, then link them. If we have a project that consists of `Class.hpp`, `Class.cpp`, and `main.cpp`, we would have to compile it as follows (assuming Linux is the platform):

```
g++ -c Class.cpp
g++ -c main.cpp
g++ Class.o main.o -o main
```

A `Makefile` allows us to provide these build instructions to `make`, so that we can simply run the `make` command to build our project. The equivalent Makefile would look like the following:

```
main: main.o Class.o
    g++ Class.o main.o -o main

Class.o: Class.cpp Class.hpp
    g++ -c Class.cpp

main.o: main.cpp Class.hpp
    g++ -c main.cpp
```

The top target of the file is `main`. By default, `make` will try to build the first target in the file. The files after the colon are the dependencies for

that target. The dependencies for `main` are `Class.o` and `main.o`. If `make` does not find those files, it will search the Makefile for a target to build them. `Class.o` has dependencies of `Class.cpp` and `Class.hpp` to be built. There are no targets to build those files, so if they do not exist, the build will fail. The `main.o` target depends on `main.cpp` and `Class.hpp` to be built. The commands underneath the targets and dependencies are run to build that target. If any of the dependencies are modified (the last modified date is more recent than the last time `make` was run), `make` will only rebuild the targets that are impacted. For example, if we modify `main.cpp`, `main.o` depends on `main.cpp` so it will be rebuilt. This then means that `main.o` has been updated, so `main` will be rebuilt as well. However, `Class.cpp` and `Class.hpp` are unaffected, so `Class.o` will not be rebuilt.

CMake

Make improved the build process for building C/C++ projects, but is far from perfect. It does not have mechanisms for ensuring dependencies exist, has a very verbose syntax, and only works on Unix-based systems. Compared to Make, which is a pure build system, CMake is a meta-build system. It generates build files (such as Makefiles) based on higher-level instructions. Whereas in Make you must track the dependencies of each file manually, CMake handles target-to-dependency resolution for you. CMake configurations are written in a file called `CMakeLists.txt`. That exact file name must be used, CMake does not automatically recognize any other file. The `CMakeLists.txt` for any project start with the following two commands:

```
cmake_minimum_required(VERSION 3.28)

project(PROJECT_NAME)
```

The `cmake_minimum_required()` command indicates the minimum version of CMake required to generate build files. Newer versions of CMake can act as older versions based on the specified version. The `project()` command indicates that CMake is being invoked to build a C/C++ project and forces it to run a series of checks to ensure that compilers are installed and detect important properties of the target machine. It also indicates to CMake the internal name it will use for the project.

Let's consider the case where we want to build an executable program from a single source code file. An executable program contains a

`main()` function and is what you've been building throughout the class. The `add_executable()` command tells CMake that you are adding an executable target to the project. The target can have multiple properties, including the output name of the executable, include directories, source code files, dependencies, and compiler flags. In `add_executable()`, we specify the name of the target executable.

```
add_executable(example_program)
```

The `target_sources()` command is used to list the source code files needed to compile the CMake file's target. In this case, we want to compile `example_program`. Let's assume the code for the program is in a file called `example.cpp` that is in the same directory as our `CMakeLists.txt` file. To add `example.cpp`, we'd write the following CMake command:

```
target_sources(example_program PRIVATE example.cpp)
```

Now, to run CMake, we use the following command (assuming we have navigated to the same directory as `CMakeLists.txt`):

```
cmake . -B build
```

This tells CMake to generate the build files for the current directory `.`, and write the generated files to a directory `-B` called `build`. CMake will automatically detect the compiler and generate the appropriate build files. To compile the program, we can run:

```
cmake --build build
```

Depending on your OS and compiler, the executable will be written directly to the `build` directory or to a subdirectory of build like `build/bin` or `build/Debug`. You can change directory to the location of the executable and run it.

Mac/Linux

```
./example_program
```

Windows

```
.\example_program.exe
```

The example code for this is in `week13_simple.zip` on Brightspace. The complete `CMakeLists.txt` for this example would be:

```
cmake_minimum_required(VERSION 3.28)

project(EXAMPLE_PROJECT)

add_executable(example_program)
target_sources(example_program PRIVATE example.cpp)
```

Libraries

Up until this point everything you've created in C++ have been executable programs. Software can also be compiled into a library. A library is a collection of functions and data types that other libraries and executables can use. `iostream`, `string`, `algorithm`, `vector`, etc. are all libraries. Libraries do not have a `main()` function.

To create a library using CMake, we can use the `add_library()` command, similar to how we used `add_executable` for a program.

```
add_library(example_library)
```

Unlike executables, where all the code it depends on is private, clients of a library do need to know the contents of the header files of the library. If you did not have access to the `iostream` header, you would never be able to include or use it. Therefore, when adding target sources to a library, we make the implementation files private and the library headers public:

```
target_sources(example_library
    PRIVATE
        example.cpp
    PUBLIC
        FILE_SET exampleHeaders
        TYPE HEADERS
        FILES
            example.hpp
)
```

The `PRIVATE` section just contains the source code for the library. The `PUBLIC` section is a bit more complicated. First, `FILE_SET` indicates the name CMake will use to refer to this set of files. It is a bit redundant here since there is only one file, but it will become more useful when we have files. `TYPE HEADERS` indicates that these are header files. `FILES` indicates the names of the header file(s). In this case, we only have one header file, `example.hpp`. We run CMake to generate the build files and compile just as before. However, rather than compiling to an executable, it will generate a compiled library file. Typically, the default is to build a static library file. On Mac/Linux, a static library file with the `.a` extension. On Windows, the `.lib` extension. There are options

that can be set to generate a shared library.

Static vs Dynamic/Shared Libraries

A static library is a fully compiled version of a library that must be integrated as part of an executable. That is, when the executable is compiled, the static library will be included as part of that executable's code. On Mac and Linux, static libraries typically have the `.a` extension. On Windows, they have the `.lib` extension. Shared (also called dynamically linked) libraries do not need to be compiled as part of an executable's machine code. Instead, when the code from the shared library is needed, the executable (while running) can request for the operating system to find that shared library and load the code it needs. This means that the overall size of the final executable is smaller. However, it also requires the client using the executable to have the shared library installed. Shared libraries have the `.so` extension on Mac and Linux, and the `.dll` extension on Windows. To build a shared library, the `SHARED` option can be set on the `add_library()` command.

```
add_library(example_static_library STATIC)
add_library(example_shared_library SHARED)
```

Building Libraries and Executables

We can also provide build instructions for libraries and executables in the same `CMakeLists.txt` file. In this example, we build a shared and static library from the header file and its implementation, and then build our executable using the static library.

```
cmake_minimum_required(VERSION 3.28)
project(EXAMPLE_PROJECT)

add_library(example_static_library STATIC)
add_library(example_shared_library SHARED)
add_executable(example_executable)

target_sources(example_static_library
    PRIVATE
        example.cpp
    PUBLIC
        FILE_SET exampleHeaders
        TYPE HEADERS
        FILES
            example.hpp
)

target_sources(example_shared_library
    PRIVATE
        example.cpp
```

```

    PUBLIC
        FILE_SET exampleHeaders
        TYPE HEADERS
        FILES
            example.hpp
    )

target_link_libraries(example_executable PRIVATE example_static_library)
target_sources(example_executable PRIVATE main.cpp)

```

Subdirectories

Larger scale projects split files into multiple subdirectories. Typically, a `CMakeLists.txt` is written for each directory. The `add_subdirectory()` command is used to tell CMake which subdirectory to search for more `CMakeLists.txt`. Below is a typical directory structure for a basic C++ project:

```

project-root
  include
    header-files
  src
    source-files

```

The `CMakeLists.txt` in the project root is used to specify the minimum CMake version, package name, and add subdirectories. Directories that only contain header files do not need a `CMakeLists.txt`, since headers are not compiled on their own. The `CMakeLists.txt` in the `src` directory specifies the libraries and/or executables to be built. There may be more nested directories within `src` or `include` for larger scale projects, but we'll assume here that this is the highest tier of nesting. The `CMakeLists.txt` for the root would be:

```

cmake_minimum_required(VERSION 3.28)
project(EXAMPLE_PROJECT)

add_subdirectory(src)

```

And for `src`:

```

add_library(example_static_library STATIC)

target_include_directories(example_static_library PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)

target_sources(example_static_library
    PRIVATE
        example.cpp
)

```

```
add_executable(example)
target_sources(example PRIVATE example.cpp)
target_link_library(example PRIVATE example_static_library)
```

The `target_include_directories` is used to specify which path(s) to search for header files. Since the headers `PUBLIC` on the library, we do not need to specify them again for the executable.